Guide Pratique Docker pour Symfony sur Windows 11

Vue d'ensemble de ce qu'on va construire

```
mon-projet-symfony/
├── docker/
│   └── apache/
│       ├── Dockerfile
│       ├── apache2.conf
│       ├── vhost.conf
│       └── php.ini
├── docker-compose.yml
├── .dockerignore
├── .env (existant, à modifier)
└── ... (ton projet Symfony existant)
```

ÉTAPE 1 — Installer Docker Desktop

Quoi faire

Installer Docker Desktop sur Windows 11.

Comment

1. Télécharger sur https://www.docker.com/products/docker-desktop
2. Lancer l'installateur `.exe`
3. Redémarrer Windows quand demandé
4. Vérifier que **Hyper-V** est activé :
   - `Panneau de configuration` → `Activer ou désactiver des fonctionnalités Windows` → cocher **Hyper-V**

Vérifier que ça marche

```
docker --version
docker-compose --version
```

Tu dois voir les numéros de version s'afficher. Si oui, Docker est prêt.

ÉTAPE 2 — Préparer ton projet Symfony

Quoi faire

Copier ton projet depuis Laragon et créer la structure des dossiers Docker.

Comment

```
# Dans PowerShell, navigue vers ton projet
cd C:\laragon\www\mon-projet-symfony

# Crée la structure des dossiers Docker
mkdir docker
mkdir docker\apache
```

ÉTAPE 3 — Créer les fichiers Docker

Fichier 1 : `docker/apache/Dockerfile`

Quoi : Recette pour construire ton conteneur PHP + Apache.

Créer le fichier `docker/apache/Dockerfile` avec ce contenu :

```
FROM php:8.4-apache

# Installer les dépendances système
RUN apt-get update && apt-get install -y \
    git \
    curl \
    zip \
    unzip \
    libfreetype-dev \
    libjpeg62-turbo-dev \
    libpng-dev \
    libxml2-dev \
    libzip-dev \
    libicu-dev \
    mariadb-client \
    && rm -rf /var/lib/apt/lists/*

# Installer les extensions PHP nécessaires à Symfony
RUN docker-php-ext-install -j$(nproc) \
    pdo \
    pdo_mysql \
    mysqli \
    xml \
    mbstring \
    zip \
    intl \
    opcache

# Configurer et installer GD (pour les images)
RUN docker-php-ext-configure gd \
    --with-freetype=/usr/include/ \
    --with-jpeg=/usr/include/ \
```

```
    && docker-php-ext-install -j$(nproc) gd

# Activer les modules Apache
RUN a2enmod rewrite headers ssl

# Copier les fichiers de configuration
COPY apache2.conf /etc/apache2/apache2.conf
COPY vhost.conf /etc/apache2/sites-available/000-default.conf
COPY php.ini /usr/local/etc/php/conf.d/custom-php.ini

RUN a2ensite 000-default

# Définir le dossier de travail
WORKDIR /var/www/html

# Installer Composer
RUN curl -sS https://getcomposer.org/installer | php -- \
    --install-dir=/usr/local/bin \
    --filename=composer

EXPOSE 80

CMD ["apache2-foreground"]
```

Fichier 2 : `docker/apache/vhost.conf`

Quoi : Dire à Apache comment servir ton projet Symfony.

Créer le fichier `docker/apache/vhost.conf` :

```
<VirtualHost *:80>
    ServerName localhost
    DocumentRoot /var/www/html/public

    <Directory /var/www/html/public>
        AllowOverride None
        Require all granted

        # Rediriger toutes les requêtes vers index.php (routing Symfony)
        RewriteEngine On
        RewriteCond %{REQUEST_FILENAME} !-f
        RewriteRule ^(.*)$ index.php [QSA,L]
    </Directory>

    # Variables d'environnement pour PHP
    SetEnv APP_ENV dev
    SetEnv APP_DEBUG 1

    # Transmettre le header Authorization à PHP (nécessaire pour l'auth)
    SetEnvIfNoCase ^Authorization$ "(.+)" HTTP_AUTHORIZATION=$1
```

```
    # Logs
    ErrorLog /var/log/apache2/error.log
    CustomLog /var/log/apache2/access.log combined
</VirtualHost>
```

Fichier 3 : `docker/apache/apache2.conf`

Quoi : Configuration globale d'Apache.

Créer le fichier `docker/apache/apache2.conf` :

```
ServerName localhost
ServerTokens Prod
ServerSignature Off

Timeout 300
KeepAlive On
MaxKeepAliveRequests 100
KeepAliveTimeout 5

HostnameLookups Off
LogLevel warn

IncludeOptional mods-enabled/*.load
IncludeOptional mods-enabled/*.conf
Include ports.conf
IncludeOptional conf-enabled/*.conf
IncludeOptional sites-enabled/*.conf
```

Fichier 4 : `docker/apache/php.ini`

Quoi : Configurer PHP pour le développement Symfony.

Créer le fichier `docker/apache/php.ini` :

```
; Afficher les erreurs en développement
display_errors = On
display_startup_errors = On
error_reporting = E_ALL
log_errors = On
error_log = /var/log/apache2/php_error.log

; Mémoire et temps d'exécution
memory_limit = 512M
max_execution_time = 300

; Upload de fichiers
```

```
file_uploads = On
upload_max_filesize = 64M
post_max_size = 64M

; Sessions
session.auto_start = 0
session.use_strict_mode = 1

; Fuseau horaire
date.timezone = Europe/Paris

; OPCache
opcache.enable = 1
opcache.revalidate_freq = 0
opcache.validate_timestamps = 1
opcache.max_accelerated_files = 20000
opcache.memory_consumption = 128
```

Fichier 5 : `docker-compose.yml`

Quoi : Chef d'orchestre — définit et relie tous tes conteneurs.

Créer le fichier `docker-compose.yml` à la racine du projet :

```
version: '3.8'

services:

  # Conteneur PHP + Apache
  web:
    build:
      context: ./docker/apache
      dockerfile: Dockerfile
    container_name: symfony-web
    restart: unless-stopped
    ports:
      - "80:80"
    volumes:
      - ./:/var/www/html          # ton code source monté dans le conteneur
    environment:
      - APP_ENV=dev
      - APP_DEBUG=1
      - DATABASE_URL=mysql://symfony:symfony_password@db:3306/symfony_db?
serverVersion=8.0
    depends_on:
      db:
        condition: service_healthy
    networks:
      - symfony-network

  # Conteneur MySQL
```

```yaml
  db:
    image: mysql:8.0.32
    container_name: symfony-db
    restart: unless-stopped
    ports:
      - "3306:3306"
    environment:
      MYSQL_ROOT_PASSWORD: root_password
      MYSQL_DATABASE: symfony_db
      MYSQL_USER: symfony
      MYSQL_PASSWORD: symfony_password
    volumes:
      - mysql-data:/var/lib/mysql    # persistance des données
    command: --default-authentication-plugin=mysql_native_password
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      timeout: 10s
      retries: 5
    networks:
      - symfony-network

  # Conteneur phpMyAdmin (optionnel, pratique)
  phpmyadmin:
    image: phpmyadmin:latest
    container_name: symfony-phpmyadmin
    restart: unless-stopped
    ports:
      - "8080:80"
    environment:
      PMA_HOST: db
      PMA_USER: symfony
      PMA_PASSWORD: symfony_password
    depends_on:
      - db
    networks:
      - symfony-network

# Volumes nommés (persistance des données MySQL)
volumes:
  mysql-data:

# Réseau interne entre les conteneurs
networks:
  symfony-network:
    driver: bridge
```

Fichier 6 : `.dockerignore`

Quoi : Exclure les fichiers inutiles du contexte de build pour aller plus vite.

Créer le fichier `.dockerignore` à la racine :

```
.git
.gitignore
node_modules
var/cache
var/log
vendor
docker
*.md
```

ÉTAPE 4 — Modifier le `.env` de Symfony

Quoi faire

Dire à Symfony de se connecter à MySQL **via le nom du conteneur** (`db`) et non `localhost`.

Modifier ton fichier `.env` existant

Trouve cette ligne :

```
# Avant (Laragon)
DATABASE_URL="mysql://root:@127.0.0.1:3306/ma_base?serverVersion=8.0"
```

Remplace par :

```
# Après (Docker) — "db" = nom du service dans docker-compose.yml
DATABASE_URL="mysql://symfony:symfony_password@db:3306/symfony_db?
serverVersion=8.0"
```

> ⚠ **Point clé** : Dans Docker, les conteneurs se parlent par leur **nom de service**, pas par `localhost`.

ÉTAPE 5 — Lancer les conteneurs

Commandes dans l'ordre

```
# 1. Se placer à la racine du projet
cd C:\laragon\www\mon-projet-symfony

# 2. Construire les images (à faire une seule fois, ou après modification des
Dockerfiles)
docker-compose build

# 3. Démarrer tous les conteneurs en arrière-plan
docker-compose up -d
```

```
# 4. Vérifier que tout tourne
docker-compose ps
```

Tu dois voir `symfony-web`, `symfony-db` et `symfony-phpmyadmin` avec le statut **Up**.

ÉTAPE 6 — Installer les dépendances et configurer Symfony

```
# Installer les dépendances Composer DANS le conteneur
docker-compose exec web composer install

# Créer la base de données
docker-compose exec web php bin/console doctrine:database:create

# Exécuter les migrations
docker-compose exec web php bin/console doctrine:migrations:migrate

# (Optionnel) Charger des fixtures
docker-compose exec web php bin/console doctrine:fixtures:load

# Vider le cache
docker-compose exec web php bin/console cache:clear
```

ÉTAPE 7 — Vérifier que tout fonctionne

Accès	URL
Ton projet Symfony	http://localhost
phpMyAdmin	http://localhost:8080

Commandes du quotidien

```
# Démarrer les conteneurs
docker-compose up -d

# Arrêter les conteneurs (données conservées)
docker-compose down

# Arrêter ET supprimer les données (reset complet)
docker-compose down -v

# Voir les logs en temps réel
docker-compose logs -f

# Voir les logs d'un seul conteneur
docker-compose logs -f web
```

```
docker-compose logs -f db

# Ouvrir un terminal DANS le conteneur PHP
docker-compose exec web bash

# Exécuter une commande Symfony
docker-compose exec web php bin/console <commande>

# Exécuter Composer
docker-compose exec web composer require <package>

# Accéder à MySQL en ligne de commande
docker-compose exec db mysql -u symfony -psymfony_password symfony_db
```

Résolution des problèmes courants

Le port 80 est déjà occupé (par Laragon)

```
# Dans docker-compose.yml, changer le port du service web
ports:
  - "8000:80"   # accès sur http://localhost:8000
```

Erreur de permissions sur `var/`

```
docker-compose exec web chown -R www-data:www-data /var/www/html/var
docker-compose exec web chmod -R 755 /var/www/html/var
```

Erreur de connexion MySQL

```
# Vérifier que le conteneur db est healthy
docker-compose ps

# Lire les logs MySQL
docker-compose logs db

# Vérifier la configuration Apache
docker-compose exec web apache2ctl configtest
```

Reconstruire après modification d'un Dockerfile

```
docker-compose build --no-cache
docker-compose up -d
```

Résumé visuel du flux

```
Navigateur
    │
    ▼ http://localhost
[Conteneur web - Port 80]
  Apache + PHP 8.4
  /var/www/html = ton code
    │
    ▼ mysql://db:3306
[Conteneur db - Port 3306]
  MySQL 8.0.32
  données dans mysql-data (volume)
    │
[Conteneur phpmyadmin - Port 8080]
  Interface graphique MySQL
```

> 💡 **Règle d'or à retenir** : Tu n'exécutes plus rien directement sur ta machine. Toutes les commandes PHP, Composer, Symfony passent par `docker-compose exec web <commande>`.